

Stack in C++ STL

Last Updated : 23 Jul, 2025



In C++, **stack** container follows LIFO (**L**ast **I**n **F**irst **O**ut) order of insertion and deletion. It means that most recently inserted element is removed first and the first inserted element will be removed last. This is done by inserting and deleting elements at only one end of the stack which is generally called the top of the stack.

Example:

```
#include <iostream>
#include <stack>
using namespace std;

int main() {
    stack<int> st;

    // Pushing elements onto the stack
    st.push(1);
    st.push(2);
    st.push(3);

    while(!st.empty()) {
        cout << st.top() << " ";
        st.pop();
    }
    return 0;
}
```

Output

```
3 2 1
```

Explanation: In the above program, we created a stack of integers **s**. The element **1**, **2** and **3** are then pushed one after other into the stack. Due to LIFO order, the elements are stored in the stack in the reverse order as they are inserted.

Syntax

Stack is defined as **std::stack** class template inside the **<stack>** header file.

```
stack<T> st;
```

where,

- **T**: Type of elements in the stack.
- **st**: Name assigned to the stack.

Declaration and Initialization

In C++, stack can be declared and initialized in two ways as shown in the below example:

```
#include <bits/stdc++.h>
using namespace std;

void print(stack<int> st) {
    while (!st.empty()) {
        cout << st.top() << " ";
        st.pop();
    }
}

int main() {

    // Declare empty stack
    stack<int> st1;

    // Insert elements into stack
    st1.push(10);
    st1.push(20);
    st1.push(30);
    st1.push(40);

    // Create another stack from st1
    stack<int> st2(st1);

    print(st1);
    cout << endl;
```

```
    print(st2);  
    return 0;  
}
```

Output

```
40 30 20 10  
40 30 20 10
```

Explanation: In the above program,

- Statement **stack<int> st1** creates an empty stack of integers. Elements are later added into **st1**.
- Statement **stack<int> st2 (s1)** creates another stack **st2** with the same elements as **st1**.

Basic Operations

Here are the basic operations that can be performed on a stack:

1. Inserting Elements

In stack, new elements can only be inserted at the top of the stack by using [push\(\)](#) method. There is no way to inserting element at any other position of the stack.

Example:

```
#include <bits/stdc++.h>  
using namespace std;  
  
int main() {  
    stack<int> st;  
  
    // Inserting element top of the stack  
    st.push(10);  
    st.push(20);  
    st.push(30);  
    st.push(40);  
  
    while(!st.empty()) {
```

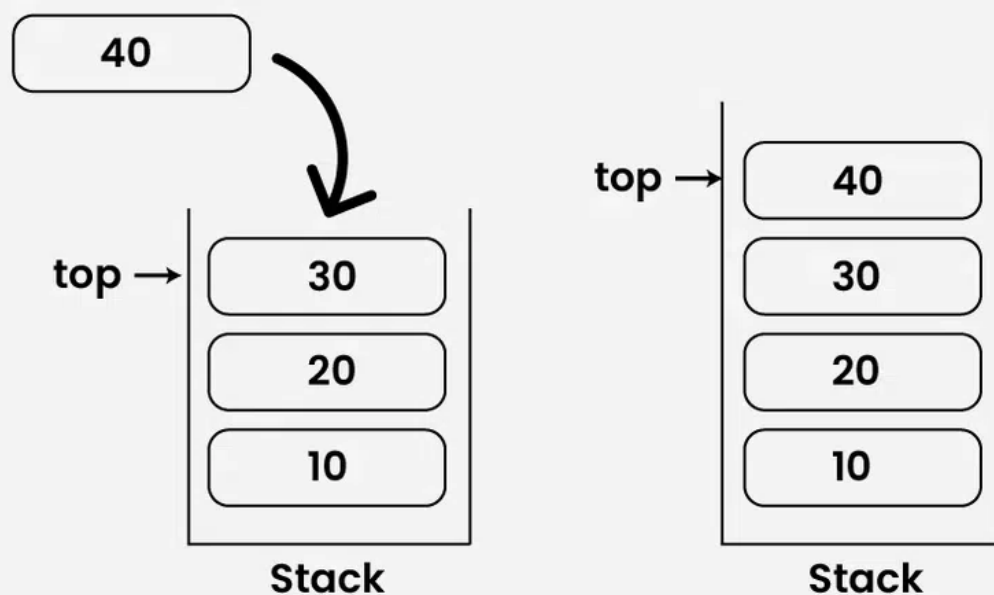
```
        cout << st.top() << " ";
        st.pop();
    }
    return 0;
}
```

Output

```
40 30 20 10
```



Push Operation in Stack



2. Accessing Elements

Only the top element of the stack can be accessed using `top()` method. We can't access any middle element of stack.

Example:

```
#include <bits/stdc++.h>
using namespace std;
```

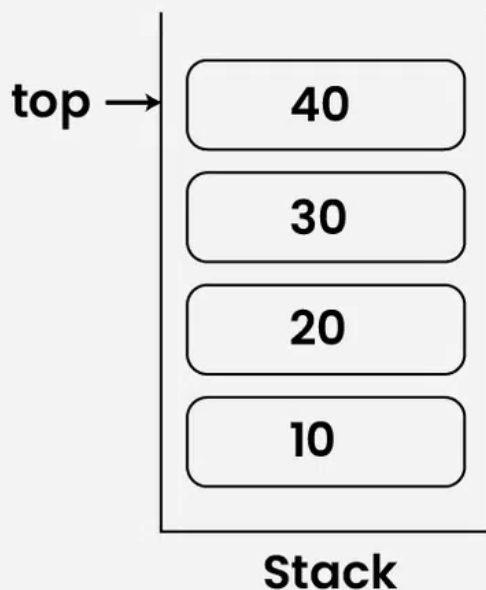
```
int main() {  
    stack<int> st;  
    st.push(10);  
    st.push(20);  
    st.push(30);  
    st.push(40);  
  
    // Accessing the top element  
    cout << st.top();  
    return 0;  
}
```

Output

40



Top or Peek Operation in Stack



Top Element = 40

3. Deleting Elements

In stack, only the top element of the stack can be deleted by using `pop()` method in one operation. To delete any other element, we have to first delete all the elements inserted after that element (i.e. elements on top of the that element).

Example:

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    stack<int> st;
    st.push(10);
    st.push(20);
    st.push(30);
    st.push(40);

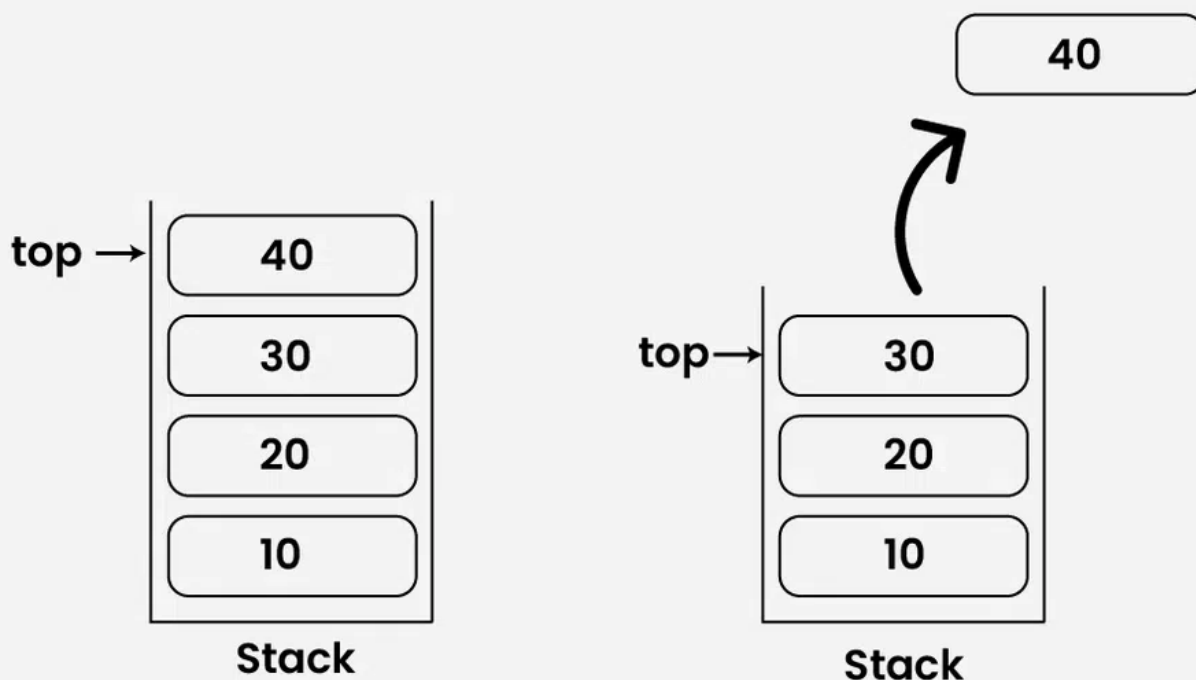
    // Deleting top element
    st.pop();

    while(!st.empty()) {
        cout << st.top() << " ";
        st.pop();
    }
    return 0;
}
```

Output

```
30 20 10
```

Pop Operation in Stack



4. Pseudo Traversal

As any element other than the top element cannot be accessed in the stack, it cannot be actually traversed. But we can create a copy of it, access the top element and delete the top. By doing this till the copy stack is empty, we can effectively traverse without modifying the original stack.

Example:

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    stack<int> st;
    st.push(10);
    st.push(20);
    st.push(30);
    st.push(40);
```

```
// Create a copy
stack<int> temp(st);

while(!temp.empty()) {
    cout << temp.top() << " ";
    temp.pop();
}
return 0;
}
```

Output

```
40 30 20 10
```

Explanation: In the above program, pseudo-traversal is done by creating a copy of the **st** to avoid modifying the original stack. The while loop prints and removes elements from temp using **top()** and **pop()** till the stack is not empty.

Time Complexity

The below table lists the time complexity of the above operations on stack:

Operation	Time Complexity
Insert an element (push)	O(1)
Delete an element (pop)	O(1)
Access top element (peek)	O(1)
Traverse the stack	O(n)

Other Common Operations

The following are some more operations on stack that helps you become more familiarize with it:

- [Check if a stack is empty](#)
- [Check the Size of the Stack](#)
- [Reverse a stack](#)
- [Clear a stack](#)
- [Copy One Stack to Another](#)
- [Swap Two Stacks](#)

Internal Working

Stack container provides the built-in implementation of the [stack data structure](#) in C++. It is implemented as container adapter built over other containers such as vectors, lists, etc. It uses the LIFO (Last In, First Out) principle to push and pop elements. The top element is always the most recently added element, and operations are limited to this end of the stack.

Note: Though stacks can be possibly implemented using vectors, its middle elements cannot be accessed due to LIFO restrictions.

All Member Functions

Following is the list of all member functions of `std::stack` class in C++:

Functions	Description
<u>push()</u>	Add an element top of the stack.
<u>push_range()</u>	Add range of elements at top of the stack.
<u>top()</u>	Access the top element of the stack.
<u>pop()</u>	Remove the top element of the stack.
<u>emplace()</u>	Add the constructs element in top of the stack.

Functions	Description
<code>size()</code>	Returns the number of elements in the stack.
<code>empty()</code>	Checks if the stack is empty.
<code>swap()</code>	Swap two stacks.

Stack in C++ STL

[Visit Course ↗](#)
[Comment](#)
[More info ▼](#)
[Advertise with us](#)

Similar Reads

C++ Programming Language

C++ is a programming language known for its fast speed, low level memory management and is often taught as first programming language. Why Learn C++?C++ is often taught as a foundational language ...

🕒 5 min read

[C++ Overview](#)

[C++ Basics](#)

[C++ Variables and Constants](#)

[C++ Data Types and Literals](#)

[C++ Operators](#)

[C++ Input/Output](#)

[C++ Control Statements](#)


C++ Functions



C++ Pointers and References



Do Not Sell or Share My Personal Information